

杭州电子科技大学

网络空间安全技术实验

实 验 报 告

学 院	网络空间安全学院
专 业	网络空间安全
班 级	Xx
学 号	Xx
学生姓名	Xx
教师姓名	xx
完成日期	2025.12.30
成 绩	

实验一 基于 Snort 的入侵检测系统实验	完成人 xx 日期 2025.12.22
一、实验任务 任务一： ➤ 学习嗅探功能：对网络接口 eth0 进行监听： ● 对实验主机的网络接口 eth0 进行监听； ● 实验主机访问 www.hdu.edu.cn； ● 在实验主机上采用详细模式在终端显示数据包网络层、链路层、应用层信息； ● 停止捕获，捕获的数据包存储到日志文件 /var/log/snort/snort.log；（存储格式可以是二进制形式，也可以是 ASCII 码形式） ➤ 学习包记录功能：对实验主机网络接口 eth0 进行监听，捕获数据包存储到日志文件中 /var/log/snort/snort.log： ● 启动 Snort，输入命令，监听主机实验主机的网络接口 eth0，当其他主机 ping 监听主机时，捕获数据包； ● 停止捕获，将捕获的数据包存储到日志文件 /var/log/snort/snort.log；（存储格式可以是二进制形式，也可以是 ASCII 码形式） ● 在实验主机上读取 snort.log 文件，查看数据包内容。 ➤ 学习入侵检测功能：添加新的规则，当有人通过任何方法以 root 用户身份在计算机外部登录计算机时，该规则将显示警报： ● 在 snort 规则集目录 ids/rules 下新建 snort 规则集文件 new.rules，对来自外部主机的、目标为当前主机的 telnet 数据包进行报警，报警消息自定义； ● 编辑 snort.conf 配置文件，使其包含 new.rules 规则集文件，具体操作如下：使用 vim (vi) 编辑器打开 snort.conf，切换至编辑模式，在最后添加新行包含规则集文件 new.rules。使用语句 include &RULE_PATH/new.rules； ● 以入侵检测方式启动 snort，进行监听，操作命令： snort -c snort.conf ； ● 用外部主机向实验主机主机系统使用 telnet 连接时，验证规则是否有效。 任务二：编写代码实现 NIDS 和 HIDS 2.1 基于 Snort 规则实现 NIDS，当有人通过任何方法以 root 用户身份在计算机外部登录	

计算机时，予以警报，具体过程如下：

- 实验的系统环境自选；
- 启动抓包工具；
- 外部主机使用 telnet 的方式访问实验主机，捕获数据包；
- 编写代码，对数据包进行解析，将网络层、链路层、应用层协议字段内容进行提取；
- 从 <https://www.snort.org/> 上下载 Snort 规则文件；（文件不能用记事本打开，可以用写字板、notepad++Pycharm 等方式打开）
- 基于 Snort 规则对数量中的异常数据包进行检测，并发出告警，告警信息发送到个人邮箱。

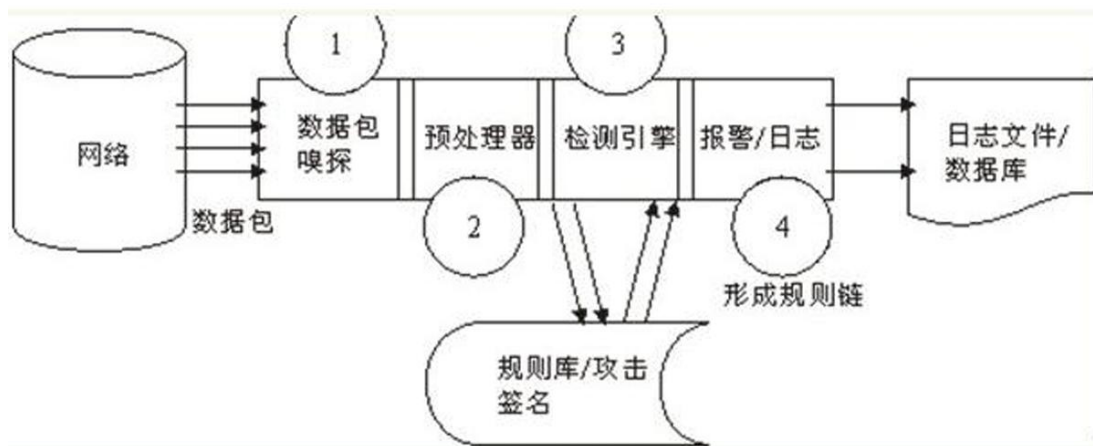
2.2 实现 HIDS，具体任务如下：

- 实验的系统环境自选；
- 显示实验主机中各个应用程序占用内存的情况；
- 统计实验主机中各个应用程序一段时间内（10 分钟）使用流量的情况；
- 监控某个文件的读写情况；
- 显示系统中的启动进程情况；

二、实验原理

Snort 是一个开源的轻量级入侵检测系统（NIDS），使用 C 语言编写，支持 windows、Linux 平台，有三种工作模式，包括：嗅探、记录数据包、入侵检测。嗅探器模式仅仅是从网络上读取数据包并作为连续不断的流显示在终端上。数据包记录器模式把数据包记录到硬盘上。网路入侵检测模式是最复杂的，而且是可配置的。可以让 snort 分析网络数据流以匹配用户定义的一些规则，并根据检测结果采取一定的动作。

Snort IDS 体系结构如图所示。



如上图所示，Snort 的结构由 4 大软件模块组成，它们分别是：

- （1）数据包嗅探模块——负责监听网络数据包，对网络进行分析；
- （2）预处理模块——该模块用相应的插件来检查原始数据包，从中发现原始数据的“行为”，如端口扫描，IP 碎片等，数据包经过预处理后才传到检测引擎；
- （3）检测模块——该模块是 Snort 的核心模块。当数据包从预处理器送过来后，检测引擎依据预先设置的规则检查数据包，一旦发现数据包中的内容和某条规则相匹配，就通知报警模块；
- （4）报警/日志模块——经检测引擎检查后的 Snort 数据需要以某种方式输出。如果检测引擎中的某条规则被匹配，则会触发一条报警，这条报警信息会通过网络、UNIXsocket、WindowsPopup(SMB)、SNMP 协议的 trap 命令传送给日志文件，甚至可以将报警传送给第三方插件（如 SnortSam），另外报警信息也可以记入 SQL 数据库。

三、实验环境/实验拓扑图

我在 Vmware 上创建了一个 Windows 10 的虚拟机，本实验均在虚拟机上完成，具体配置如下图：

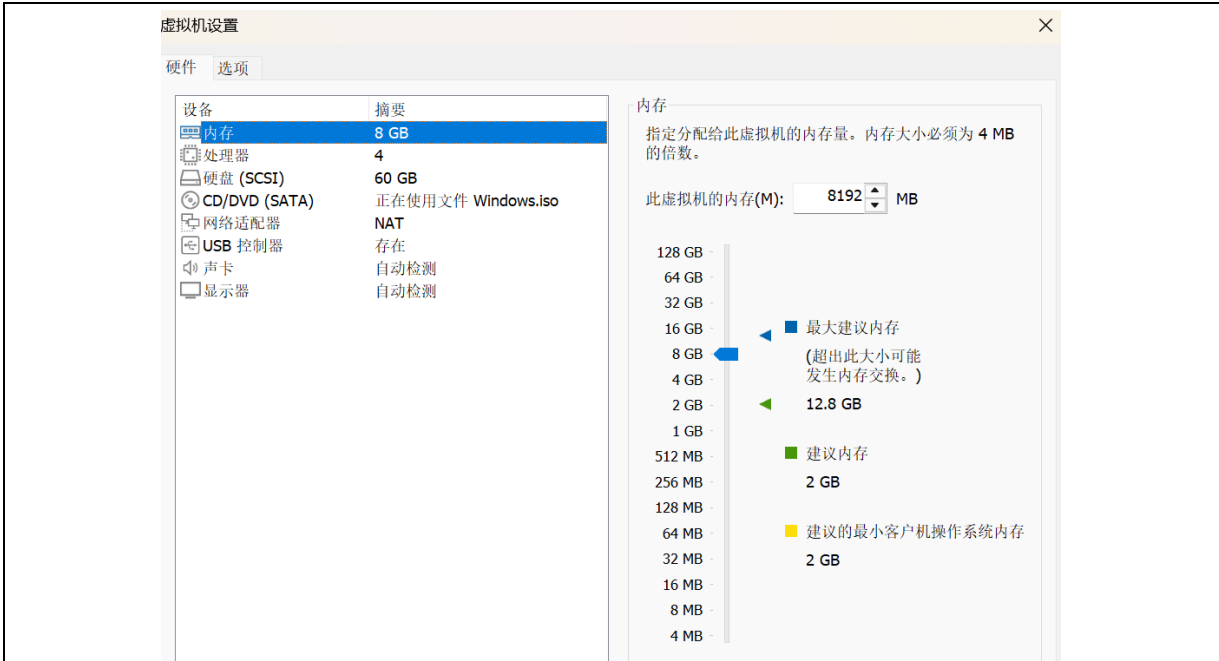


图 1 虚拟机配置

本实验所需用到的环境以及软件有：

WinPcap(4.1.3 版本)

Snort（2.9.16 x86 版本）

```
C:\Users\30682>cd C:/snort
C:\snort>cd bin
C:\snort\bin>snort -V

o''~
'''~
-*> Snort! <*-
Version 2.9.16-WIN32 GRE (Build 118)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.3

C:\snort\bin>
```

Python（3.10.2 版本）

Prcharm（2025.3.1 版本）

四、主要操作步骤及实验结果记录

实现嗅探功能：

1、在 Snort 的 bin 目录下打开 cmd，输入 snort -W 查看当前网卡，可以看到网卡编号为 1。

```

C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.19045.6456]
(c) Microsoft Corporation。保留所有权利。

C:\snort\bin>snort -W

~> Snort! <*-
Version 2.9.16-WIN32 GRE (Build 118)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.3

Index  Physical Address      IP Address      Device Name      Description
-----
1      00:0C:29:19:02:06      0000:0000:fe80:0000:0000:0000:0a11:2980 \Device\NPF_{C793FC0A-DA56-4F84-BC9D-C3C50C2989E3}
3      Intel(R) 82574L Gigabit Network Connection

C:\snort\bin>

```

图 1 查看网卡编号

2、知道网卡编号以后，接下来在实验主机上采用详细模式进行嗅探，在终端显示数据包网络层、链路层、应用层信息，并将捕获到的数据包存储到日志文件 C:\snort\log 文件夹中，输入命令 `snort -i 1 -d -v -e -l C:\snort\log`，如图 2 所示。

- -i 1: 指定监听的网卡为 1 号网卡；
- -d: 显示应用层信息（TCP/UDP 载荷，比如 HTTP、DNS 等应用层数据）
- -v: 详细显示网络层（IP）+ 传输层（TCP/UDP）信息；
- -e: 额外显示链路层（以太网帧）信息；
- -l C:\snort\log: 指定日志保存到指定目录；

```

C:\snort\bin>snort -i 1 -d -v -e -l C:\snort\log
Running in packet logging mode

==== Initializing Snort ====
Initializing Output Plugins!
Log directory = C:\snort\log
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{C793FC0A-DA56-4F84-BC9D-C3C50C2989E3}".
Decoding Ethernet

==== Initialization Complete ====

~> Snort! <*-
Version 2.9.16-WIN32 GRE (Build 118)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.3

```

图 2 实现嗅探功能

3、期间，我打开浏览器访问了 www.hdu.edu.cn 网站，然后关闭嗅探，如图 3，snort.log.1766904720 就

是捕捉到的包，这里存储格式为二进制形式，无法查看，我使用 `snort -r C:\snort\log\snort.log.1766904720 -d -v -e > C:\snort\log\ziutun.txt` 将其转换成 ASCII 码形式，然后用记事本打开查看。

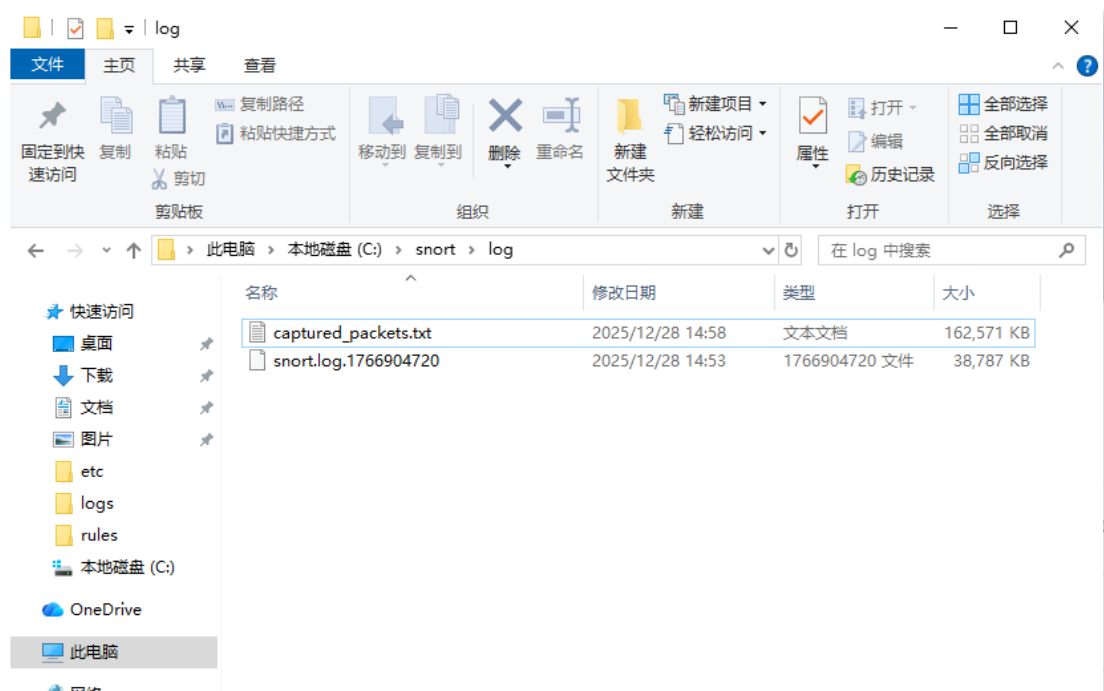


图 3 捕获到的数据包

4、包分析：如图所示，成功捕捉到了访问 www.hdu.edu.cn 的链路层、网络层、应用层的数据包，如图 4 所示，有许多 TCP 和 UDP 的包，本地 IP 地址为 192.168.146.128。

链路层信息：能看到 00:0C:29:19:02:06 -> 00:50:56:E2:15:C9（源 / 目标 MAC 地址）、type:0x800（IP 帧类型）。

网络层信息：能看到 192.168.146.128 -> 192.168.146.2（源 / 目标 IP）、TTL:128、ID:38663 等。

应用层信息：53 端口是 DNS 协议（域名解析的应用层协议），正是访问 www.hdu.edu.cn 时，系统先发起的 DNS 解析请求（把域名转成 IP）；日志里清晰显示 03 77 77 77 03 68 64 75 03 65 64 75 02 63 6E（对应 ASCII 就是 www.hdu.edu.cn），这是应用层的 DNS 请求数据。

```

12/28-14:52:52.218720 00:0C:29:19:02:06 -> 00:50:56:E2:15:C9 type:0x800 len:0x4A
192.168.146.128:63659 -> 192.168.146.2:53 UDP TTL:128 TOS:0x0 ID:38663 IpLen:20 DgmLen:60
Len: 32
E3 16 01 00 00 01 00 00 00 00 00 03 77 77 77 .....www
03 68 64 75 03 65 64 75 02 63 6E 00 00 41 00 01 .hdu.edu.cn..A..

=====

12/28-14:52:52.218897 00:0C:29:19:02:06 -> 00:50:56:E2:15:C9 type:0x800 len:0x4A
192.168.146.128:62354 -> 192.168.146.2:53 UDP TTL:128 TOS:0x0 ID:38664 IpLen:20 DgmLen:60
Len: 32
21 DC 01 00 00 01 00 00 00 00 00 00 03 77 77 77 !.....www
03 68 64 75 03 65 64 75 02 63 6E 00 00 01 00 01 .hdu.edu.cn.....

=====

12/28-14:52:52.227862 00:0C:29:19:02:06 -> 00:50:56:E2:15:C9 type:0x800 len:0x4A
192.168.146.128:64141 -> 192.168.146.2:53 UDP TTL:128 TOS:0x0 ID:38665 IpLen:20 DgmLen:60
Len: 32
09 10 01 00 00 01 00 00 00 00 00 03 77 77 77 .....www
03 68 64 75 03 65 64 75 02 63 6E 00 00 01 00 01 .hdu.edu.cn.....

```

图 4 捕捉到的有关 www.hdu.edu.cn 的包

学习包记录功能:

5、嗅探网卡 1: snort -i 1 -l C:\snort\log, 然后使用本地主机用 ping 该虚拟机（先在虚拟机上使用 ipconfig 查询虚拟机本地 ip 地址为 192.168.146.128, 主机本地 ip 地址为 192.168.146.1），可以看到 snort 嗅探到了 ICMP 包。

```

C:\snort\bin>snort -i 1 -l C:\snort\log
Running in packet logging mode

==== Initializing Snort ====
Initializing Output Plugins!
Log directory = C:\snort\log
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{C793FC0A-DA56-4F84-BC9D-C3C50C2989E3}".
Decoding Ethernet

==== Initialization Complete ====

o''~
',,,)
-*)> Snort! <*-
Version 2.9.16-WIN32 GRE (Build 118)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.2

```

图 5 嗅探网卡 1


```

C:\Users\86137>ping 192.168.146.128

正在 Ping 192.168.146.128 具有 32 字节的数据:
来自 192.168.146.128 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.146.128 的回复: 字节=32 时间=3ms TTL=128
来自 192.168.146.128 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.146.128 的回复: 字节=32 时间<1ms TTL=128

192.168.146.128 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 3ms, 平均 = 0ms

```

图 6 本地主机 ping 虚拟机

```

WARNING: No preprocessors configured for policy 0.
12/27-17:19:06.772930 192.168.146.1 -> 192.168.146.128
ICMP TTL:128 TOS:0x0 ID:59268 IpLen:20 DgmLen:60
Type:8 Code:0 ID:1 Seq:3 ECHO
+++++

WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
12/27-17:19:11.382506 192.168.146.1 -> 192.168.146.128
ICMP TTL:128 TOS:0x0 ID:59269 IpLen:20 DgmLen:60
Type:8 Code:0 ID:1 Seq:4 ECHO
+++++

WARNING: No preprocessors configured for policy 0.
12/27-17:19:16.371517 192.168.146.1 -> 192.168.146.128
ICMP TTL:128 TOS:0x0 ID:59270 IpLen:20 DgmLen:60
Type:8 Code:0 ID:1 Seq:5 ECHO
+++++

```

图 7 捕获到了 ICMP 包

6、ctrl + c 终止后，可查看数据包分析，如图 8 有 ICMP 包被捕获，使用 snort -r C:\snort\log\snort.log.1766908315 -d -v -e > C:\snort\log\baojilu.txt 将原始捕获到的包转成 ASCII 码形式，然后记事本打开查看，如图 9 所示，可以看到这些数据包是本地机（192.168.146.1）与虚拟机（192.168.146.128）之间的 ICMP 协议通信包：前一个是本地机发起的 ping 请求（Type8, ECHO），后一个是虚拟机回复的 ping 响应（Type0, ECHO REPLY），包中包含双方的 MAC 地址、TTL 等网络层信息，以及“abcdefghijklmnop”这类 ping 测试用的明文数据，说明两者的 ping 通信已被 Snort 成功捕获并记录。

```

=====
Breakdown by protocol (includes rebuilt packets):
  Eth:      74 (100.000%)
  VLAN:     0 (  0.000%)
  IP4:      54 ( 72.973%)
  Frag:     0 (  0.000%)
  ICMP:     8 ( 10.811%)
  UDP:      3 (  4.054%)
  TCP:     40 ( 54.054%)
  IP6:      0 (  0.000%)
  IP6 Ext:  0 (  0.000%)
  IP6 Opts: 0 (  0.000%)
  Fragment: 0 (  0.000%)
=====

```

图 8 捕获到的包

```

12/28-15:52:28.978795 00:50:56:C0:00:08 -> 00:0C:29:19:02:06 type:0x800 len:0x4A
192.168.146.1 -> 192.168.146.128 ICMP TTL:128 TOS:0x0 ID:36955 IpLen:20 DgmLen:60
Type:8 Code:0 ID:1 Seq:39 ECHO
61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 abcdefghijklmnop
71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 qrstuvwabcdefghi

=====

12/28-15:52:28.978901 00:0C:29:19:02:06 -> 00:50:56:C0:00:08 type:0x800 len:0x4A
192.168.146.128 -> 192.168.146.1 ICMP TTL:128 TOS:0x0 ID:51911 IpLen:20 DgmLen:60
Type:0 Code:0 ID:1 Seq:39 ECHO REPLY
61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 abcdefghijklmnop
71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 qrstuvwabcdefghi

=====

12/28-15:52:29.988259 00:50:56:C0:00:08 -> 00:0C:29:19:02:06 type:0x800 len:0x4A
192.168.146.1 -> 192.168.146.128 ICMP TTL:128 TOS:0x0 ID:36956 IpLen:20 DgmLen:60
Type:8 Code:0 ID:1 Seq:40 ECHO
61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 abcdefghijklmnop
71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69 qrstuvwabcdefghi

```

图 9 ICMP 包

学习入侵检测功能:

7、在 snort 规则集目录 C:\snort\rules 下新建 snort 规则集文件 new.rules，对来自外部主机的、目标为当前主机的 telnet 数据包进行报警，具体规则如下：

```

alert tcp any any -> 192.168.146.128 23 (flags:S; msg:"Telnet 23 Port Connection Attempt Detected"; sid:1000001; rev:1;)

```

该规则检测外部主机向实验主机 23 端口（telnet）发起的 TCP 连接请求，触发报警，192.168.146.128 是虚拟机 IP 地址，端口 23 为 telnet 端口。

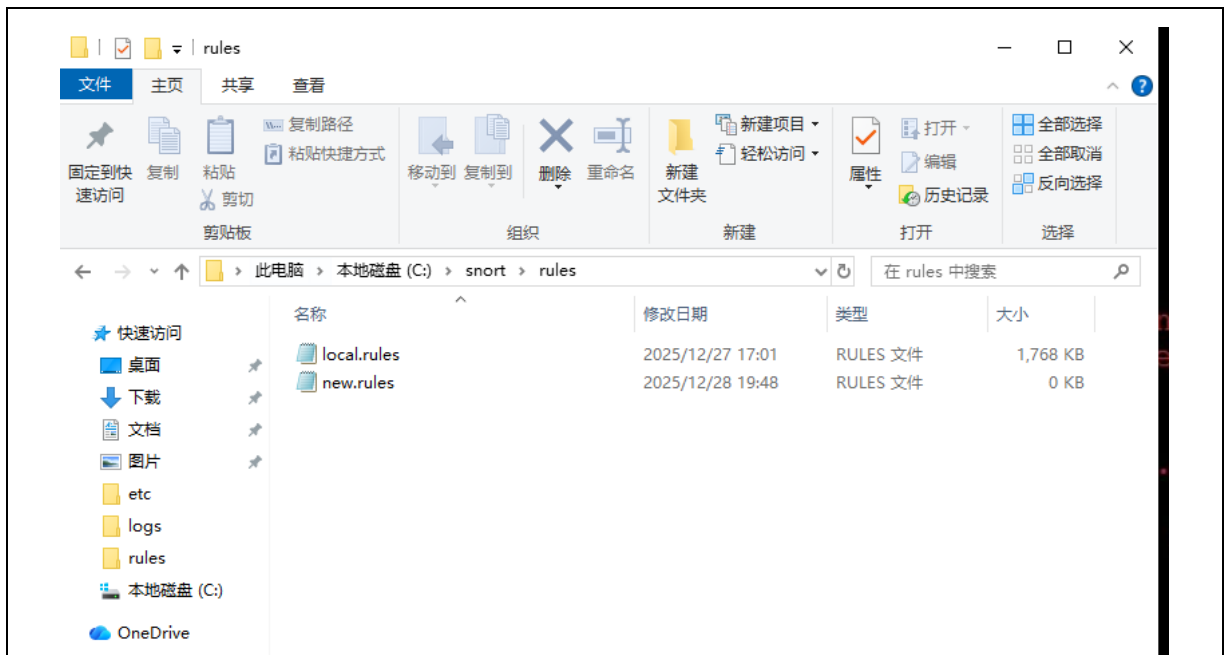


图 10 创建新规则

8、编辑 snort.conf 配置文件，使其包含 new.rules 规则集，具体操作如下：找到 C:\snort\etc\snort.conf，在最后添加新行包含规则集文件 new.rules。使用语句 include &RULE_PATH/new.rules；这里我先把其他的规则注释掉了，防止出现矛盾。这里的的\$RULE_PATH 指的是 C:\snort\rules, 要在前面设置好路径指代

*snort.conf - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

Step #7: Customize your rule set

For more information, see Snort Manual, Writing Snort Rules

#

NOTE: All categories are enabled in this conf file

#####

site specific rules

#include \$RULE_PATH/local.rules

include &RULE_PATH/new.rules

|

#include \$RULE_PATH/app-detect.rules

#include \$RULE_PATH/attack-responses.rules

#include \$RULE_PATH/backdoor.rules

#include \$RULE_PATH/bad-traffic.rules

#include \$RULE_PATH/blacklist.rules

#include \$RULE_PATH/botnet-cnc.rules

#include \$RULE_PATH/browser-chrome.rules

图 11 添加新规则

```
# Path to your rules files (this can be a relative path)
# Note for Windows users: You are advised to make this an absolute path
# such as: c:\snort\rules
var RULE_PATH C:\snort\rules
var SO_RULE_PATH C:\snort\so_rules
var PREPROC_RULE_PATH C:\snort\preproc_rules
```

图 12 定义规则路径

9、我尝试了许多次修改 snort.conf 文件，但是一直报错，有文件未配置和写法不匹配等问题，最终尝试直接在命令行输入 snort -i 1 -A console -c C:\Snort\rules\new.rules，直接使用我写的那一条规则去检测，不加载配置文件了，如图已成功启动 snort。

-A console: -A (alert) 设置告警输出方式为“控制台(console)”，当 Snort 检测到匹配规则的异常数据包时，会实时在命令行终端打印告警信息。

```
Verifying Preprocessor Configurations:
[ Port Based Pattern Matching Memory ]
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\Device\NPF_{C793FC0A-DA56-4F84-BC9D-C3C50C2989E3}"
Decoding Ethernet

--- Initialization Complete ---

o''~
o,,,~
-*> Snort! <*-
Version 2.9.16-WIN32 GRE (Build 118)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using PCRE version: 8.10 2010-06-25
Using ZLIB version: 1.2.3

Commencing packet processing (pid=9000)
WARNING: No preprocessors configured for policy 0.
```

图 13 启动 snort

10、保持 Snort 窗口打开，本地机（外部主机）管理员 CMD 执行 telnet 192.168.146.128 23，如图 14 所示，然后回到 snort 窗口，可以发现报警消息，如图 15 所示。从结果来看，这个 Snort 入侵检测实验已成功完成：本地机向虚拟机（192.168.146.128）23 端口发起 telnet 连接（虽因无 Telnet 服务显示“连接失败”），但 Snort 以 IDS 模式成功捕获了这些 TCP 连接请求包，终端中多次输出“Telnet 23 Port Connection Attempt Detected”的自定义报警信息，伴随的“No preprocessors configured”是 IDS 模式下的正常提醒，不影响检测功能。

```
C:\Users\86137>telnet 192.168.146.128 23
正在连接192.168.146.128...无法打开到主机的连接。 在端口 23: 连接失败

C:\Users\86137>|
```

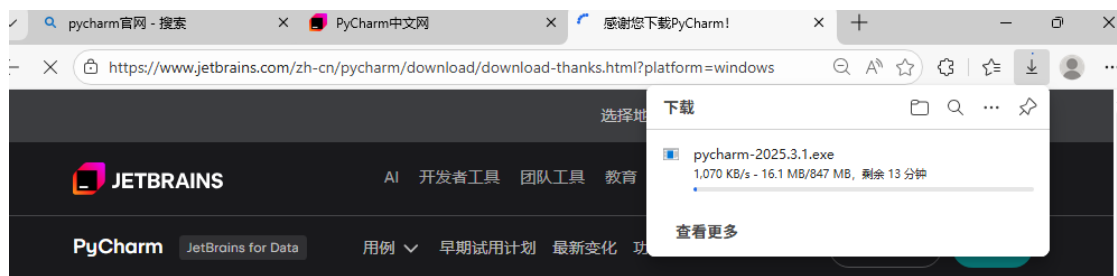
图 14 外部主机 telnet

```
WARNING: No preprocessors configured for policy 0.
12/29-10:19:42.090202  [**] [1:1000001:1] Telnet 23 Port Connection Attempt Detected [**] [Priority: 0] {TCP} 192.168.146.1:62328 -> 192.168.146.128:23
WARNING: No preprocessors configured for policy 0.
12/29-10:19:43.100619  [**] [1:1000001:1] Telnet 23 Port Connection Attempt Detected [**] [Priority: 0] {TCP} 192.168.146.1:62328 -> 192.168.146.128:23
WARNING: No preprocessors configured for policy 0.
12/29-10:19:45.102710  [**] [1:1000001:1] Telnet 23 Port Connection Attempt Detected [**] [Priority: 0] {TCP} 192.168.146.1:62328 -> 192.168.146.128:23
WARNING: No preprocessors configured for policy 0.
12/29-10:19:49.115931  [**] [1:1000001:1] Telnet 23 Port Connection Attempt Detected [**] [Priority: 0] {TCP} 192.168.146.1:62328 -> 192.168.146.128:23
WARNING: No preprocessors configured for policy 0.
12/29-10:19:57.127721  [**] [1:1000001:1] Telnet 23 Port Connection Attempt Detected [**] [Priority: 0] {TCP} 192.168.146.1:62328 -> 192.168.146.128:23
```

图 15 显示报警信息

基于 Snort 规则实现 NIDS，当有人通过任何方法以 root 用户身份在计算机外部登录计算机时，予以警报：

11、准备实验的前置环境，python 我已经提前安装好了，版本为 3.10.2，pycharm 去官网下载，版本为 2025.3.1，另外还需要 Scapy 库，用于 Python 解析数据包，打开 pycharm，在终端输入 `pip install scapy --trusted-host pypi.org --trusted-host files.pythonhosted.org` 即可，`--trusted-host` 是跳过 SSL 证书验证，直接下载包，避开 certifi 路径错误问题。



感谢您下载 PyCharm!

下载即将开始。如果没有开始，请使用此[直接链接](#)。

图 16 下载 pycharm

```
PS C:\snort> pip install scapy --trusted-host pypi.org --trusted-host files.pythonhosted.org
Collecting scapy
  Downloading scapy-2.7.0-py3-none-any.whl (2.6 MB)
    | 2.6 MB 2.2 MB/s
Installing collected packages: scapy
Successfully installed scapy-2.7.0
```

图 17 下载 scapy 库

12、将之前捕捉到的 telnet 数据包复制一份，然后命名为 telnet.pcap，打开 pycharm，创建一个 py 文件，命名为 parse_packet.py，输入如图所示的代码，接着在终端输入图 19 所示的代码。

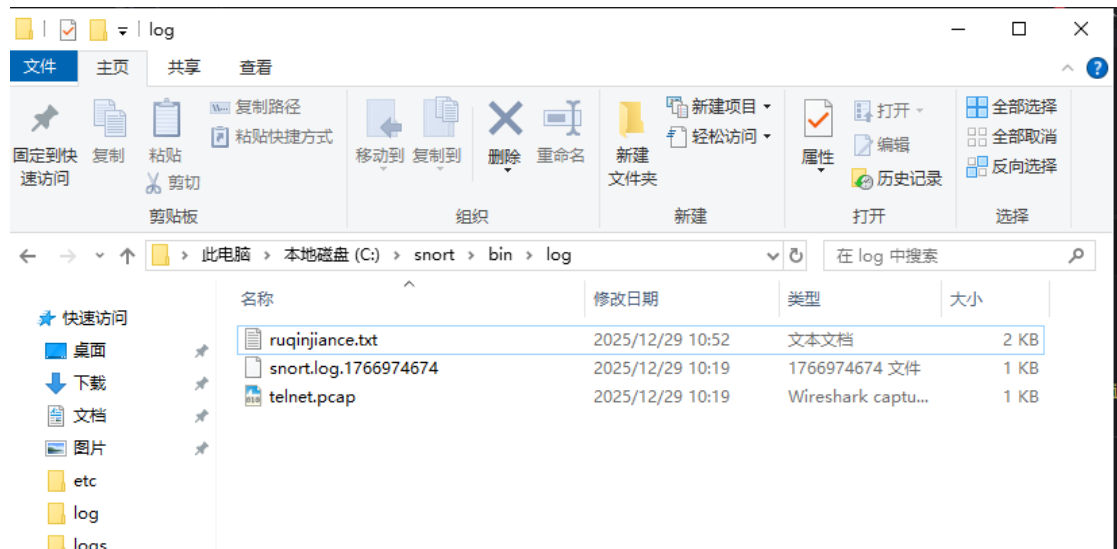


图 18 捕获数据包

```
parse_packet.py
1 from scapy.all import *
2
3 # 读取捕获的telnet数据包文件
4 packets = rdpcap("C:/snort/bin/log/telnet.pcap")
5
6 # 遍历每个数据包，提取三层字段
7 for i, pkt in enumerate(packets):
8     print(f"===== 解析第 {i+1} 个数据包 =====")
9     # 1. 提取链路层字段 (Ethernet)
10    if Ether in pkt:
11        print(f"【链路层】源MAC: {pkt[Ether].src} | 目标MAC: {pkt[Ether].dst} | 帧类型: {hex(pkt[Ether].type)}")
12    # 2. 提取网络层字段 (IP+TCP)
13    if IP in pkt and TCP in pkt:
14        print(f"【网络层】源IP: {pkt[IP].src} | 目标IP: {pkt[IP].dst} | TTL: {pkt[IP].ttl}")
15        print(f"【传输层】源端口: {pkt[TCP].sport} | 目标端口: {pkt[TCP].dport} | TCP标志位: {pkt[TCP].flags}")
16    # 3. 提取应用层字段 (telnet, 若有数据则显示)
17    if Raw in pkt:
18        app_data = pkt[Raw].load.hex() # 转16进制，避免乱码
19        print(f"【应用层】telnet数据 (16进制): {app_data[:50]}..." # 只显示前50位
20        print("-" * 50)
```

图 19 编写数据包解析代码

13、在终端运行 `python parse_packet.py`，得到如下图所示的包解析结果，有链路层的 MAC 地址信息、网络层的 IP 信息，以及传输层的端口信息等，但 telnet 过程可能并未涉及应用层信息，没有应用层的结果，代码中是有写的。

```
PS C:\snort> python parse_packet.py
===== 解析第 1 个数据包 =====
【链路层】源MAC: 00:50:56:c0:00:08 | 目标MAC: 00:0c:29:19:02:06 | 帧类型: 0x800
【网络层】源IP: 192.168.146.1 | 目标IP: 192.168.146.128 | TTL: 128
【传输层】源端口: 62328 | 目标端口: 23 | TCP标志位: S
-----
===== 解析第 2 个数据包 =====
【链路层】源MAC: 00:50:56:c0:00:08 | 目标MAC: 00:0c:29:19:02:06 | 帧类型: 0x800
【网络层】源IP: 192.168.146.1 | 目标IP: 192.168.146.128 | TTL: 128
【传输层】源端口: 62328 | 目标端口: 23 | TCP标志位: S
-----
===== 解析第 3 个数据包 =====
【链路层】源MAC: 00:50:56:c0:00:08 | 目标MAC: 00:0c:29:19:02:06 | 帧类型: 0x800
【网络层】源IP: 192.168.146.1 | 目标IP: 192.168.146.128 | TTL: 128
【传输层】源端口: 62328 | 目标端口: 23 | TCP标志位: S
-----
===== 解析第 4 个数据包 =====
【链路层】源MAC: 00:50:56:c0:00:08 | 目标MAC: 00:0c:29:19:02:06 | 帧类型: 0x800
【网络层】源IP: 192.168.146.1 | 目标IP: 192.168.146.128 | TTL: 128
【传输层】源端口: 62328 | 目标端口: 23 | TCP标志位: S
-----
===== 解析第 5 个数据包 =====
【链路层】源MAC: 00:50:56:c0:00:08 | 目标MAC: 00:0c:29:19:02:06 | 帧类型: 0x800
【网络层】源IP: 192.168.146.1 | 目标IP: 192.168.146.128 | TTL: 128
【传输层】源端口: 62328 | 目标端口: 23 | TCP标志位: S
-----
```

图 20 包解析结果

14、在 <https://www.snort.org/>上下载 snort 规则文件并解压，下载的是 snort v2.9 的社区版本。

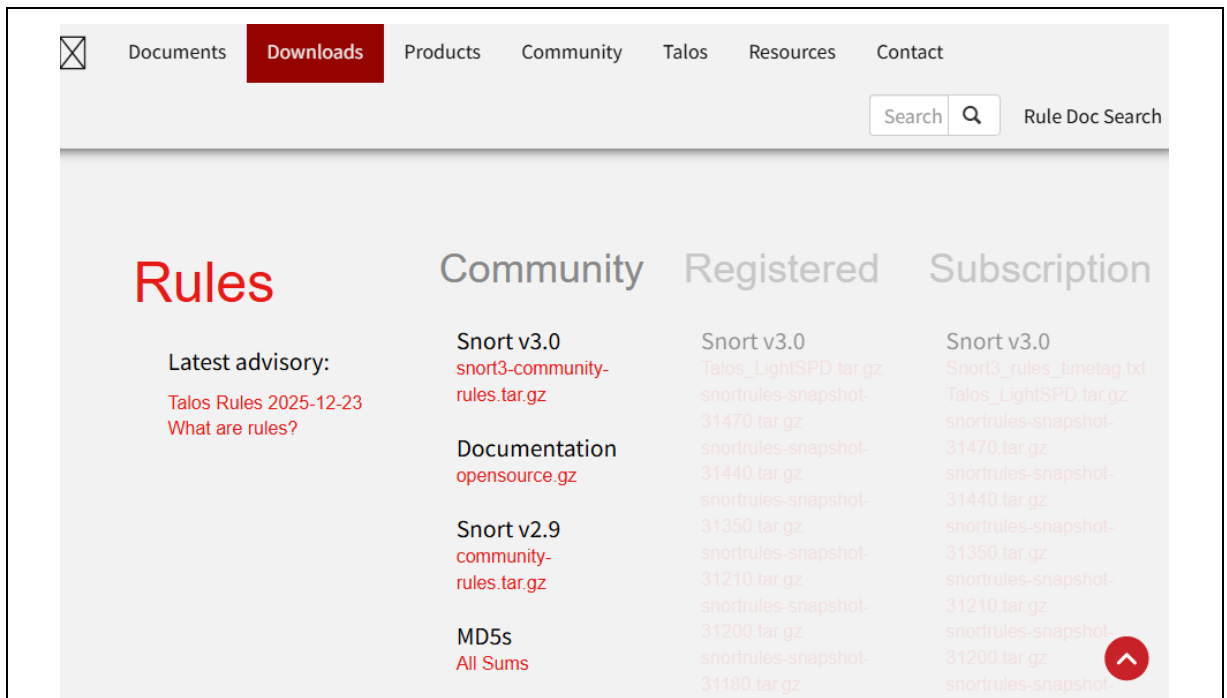


图 21 下载规则文件

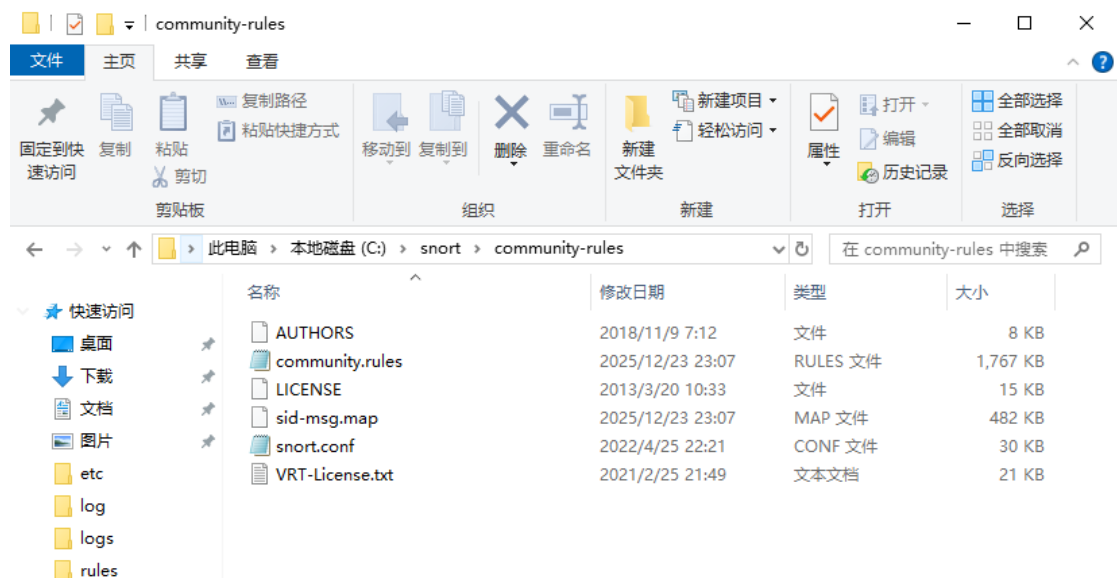


图 22 解压规则文件

15、在 community.rules 的最后一行再加一个自定义规则 alert tcp any any -> 192.168.146.128 23 (msg:"COMMUNITY ALERT: Telnet Root Login Attempt"; content:"root"; nocase; flags:S; sid:1000003; rev:1;)


```

alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS (msg:"SERVER-WEBAPP Atlassian"
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS (msg:"SERVER-WEBAPP Atlassian"
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS (msg:"SERVER-WEBAPP Atlassian"
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS (msg:"SERVER-WEBAPP Microsoft
# alert tcp $HOME_NET any -> $EXTERNAL_NET $HTTP_PORTS (msg:"MALWARE-CNC Win.Dro
alert tcp any any -> any $HTTP_PORTS (msg:"SERVER-WEBAPP Atlassian Confluence remote co
alert tcp any any -> any $HTTP_PORTS (msg:"SERVER-WEBAPP Atlassian Confluence remote co
alert tcp any any -> 192.168.146.128 23 (msg:"COMMUNITY ALERT: Telnet Root Login Attempt"

```

图 23 添加规则

16、因为后面要将告警信息发送到个人邮箱，我使用的是 qq 邮箱，这里需要打开 qq 邮箱的 pop3/smtp 服务，并获取授权码，后续会将该授权码放到代码中。



图 24 打开 qq 邮箱服务

17、接着在虚拟机打开 cmd，启动 snort 服务抓包，在外部主机使用 telnet 尝试连接，去触发规则，抓包完成后保存为 telnet.pcap 并放置于根目录下，运行如下图所示的代码，对数据包进行检测，将告警信息发到个人邮箱。

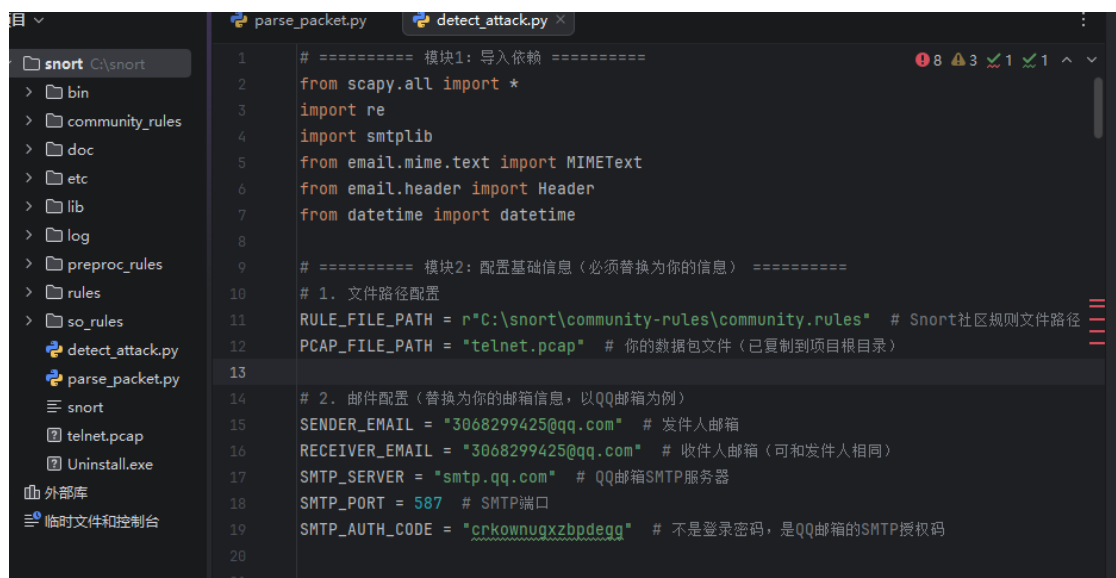


图 25 编写告警代码

18、结果展示，如图 26 所示，社区版的规则中总共有 562 条 snort 规则，然后捕获到的数据包 telnet.pcap 共有 5 个数据包，这些数据包既匹配了我自定义的“Telnet Root 登录尝试”规则，也命中了社区规则中针对 SMB 内存泄露、恶意软件 C2 连接、OpenSSL 溢出等安全风险的检测逻辑，最后告警邮件也成功发送到我的个人 qq 邮箱，如图 27 所示。

```
PS C:\snort> python detect_attack.py
正在解析Snort社区规则...
✅ 成功解析 562 条Snort规则

正在读取数据包文件...
✅ 成功读取 5 个数据包

正在检测异常数据包...
▲ 检测到异常包！匹配规则：OS-WINDOWS Microsoft Windows SMB possible leak of kernel heap memory
▲ 检测到异常包！匹配规则：MALWARE-CNC Unix.Malware.Drovorub cnc inbound connection attempt
▲ 检测到异常包！匹配规则：SERVER-OTHER OpenSSL x509 crafted email address buffer overflow attempt
▲ 检测到异常包！匹配规则：COMMUNITY ALERT: Telnet Root Login Attempt
▲ 检测到异常包！匹配规则：OS-WINDOWS Microsoft Windows SMB possible leak of kernel heap memory
▲ 检测到异常包！匹配规则：MALWARE-CNC Unix.Malware.Drovorub cnc inbound connection attempt
▲ 检测到异常包！匹配规则：SERVER-OTHER OpenSSL x509 crafted email address buffer overflow attempt
▲ 检测到异常包！匹配规则：COMMUNITY ALERT: Telnet Root Login Attempt
▲ 检测到异常包！匹配规则：OS-WINDOWS Microsoft Windows SMB possible leak of kernel heap memory
▲ 检测到异常包！匹配规则：MALWARE-CNC Unix.Malware.Drovorub cnc inbound connection attempt
▲ 检测到异常包！匹配规则：SERVER-OTHER OpenSSL x509 crafted email address buffer overflow attempt
▲ 检测到异常包！匹配规则：COMMUNITY ALERT: Telnet Root Login Attempt
▲ 检测到异常包！匹配规则：OS-WINDOWS Microsoft Windows SMB possible leak of kernel heap memory
▲ 检测到异常包！匹配规则：MALWARE-CNC Unix.Malware.Drovorub cnc inbound connection attempt
▲ 检测到异常包！匹配规则：SERVER-OTHER OpenSSL x509 crafted email address buffer overflow attempt
▲ 检测到异常包！匹配规则：COMMUNITY ALERT: Telnet Root Login Attempt
▲ 检测到异常包！匹配规则：OS-WINDOWS Microsoft Windows SMB possible leak of kernel heap memory
▲ 检测到异常包！匹配规则：MALWARE-CNC Unix.Malware.Drovorub cnc inbound connection attempt
▲ 检测到异常包！匹配规则：SERVER-OTHER OpenSSL x509 crafted email address buffer overflow attempt
▲ 检测到异常包！匹配规则：COMMUNITY ALERT: Telnet Root Login Attempt

总计检测到 20 个异常数据包
✅ 告警邮件已成功发送！
```

图 26 异常检测

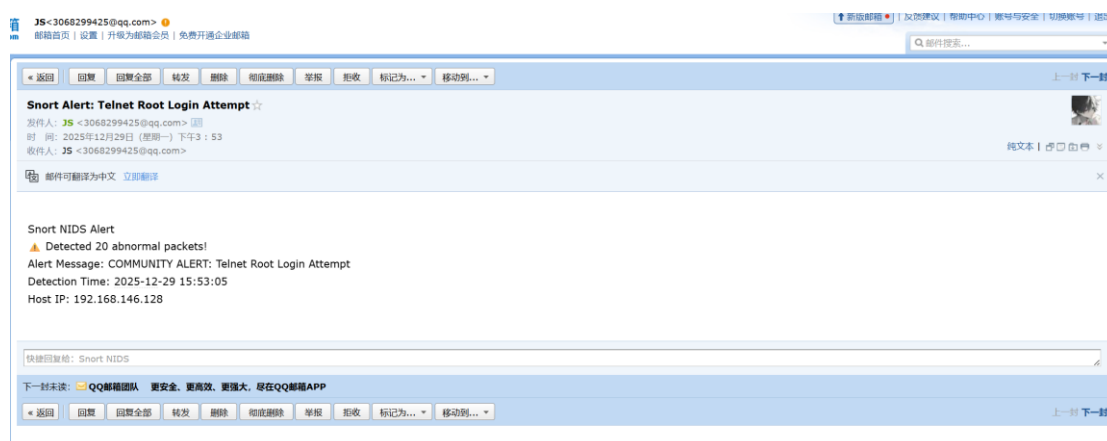
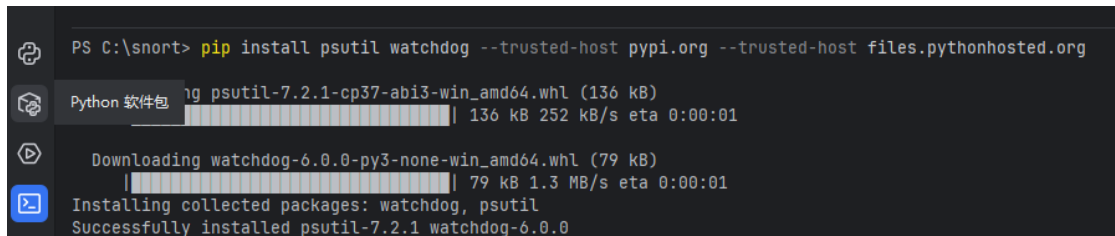


图 27 告警信息发送到个人邮箱

实现 HIDS:

19、前置实验环境准备：打开 pycharm，终端输入 `pip install psutil watchdog --trusted-host pypi.org --trusted-host files.pythonhosted.org`，信息与下载如下：

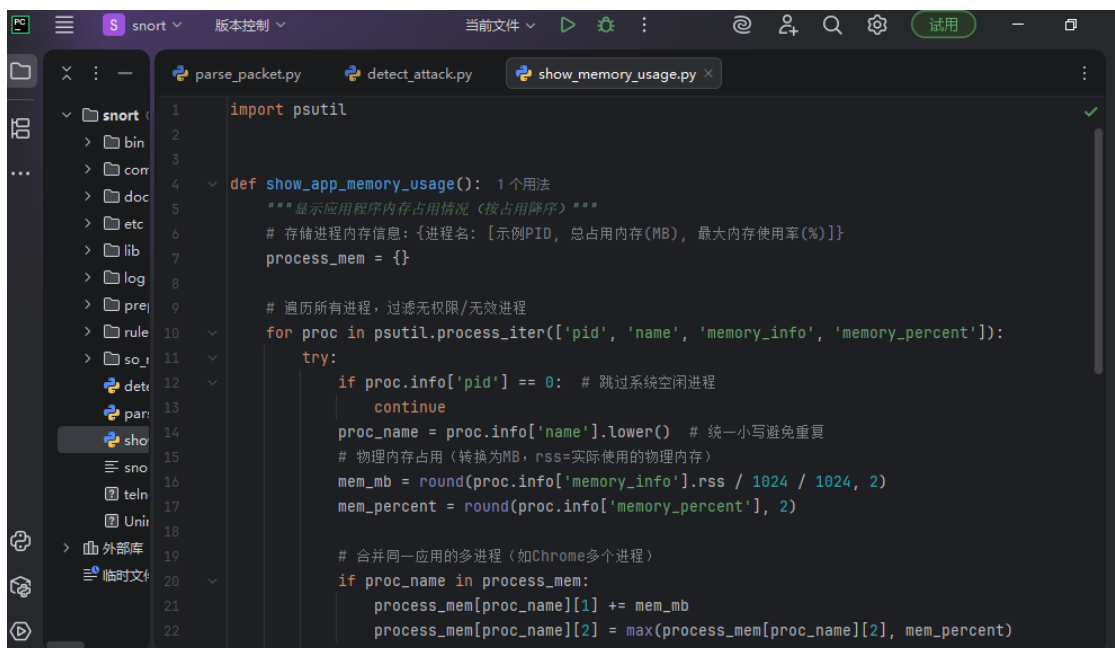
- psutil：核心库，用于获取进程、内存、网络 IO、启动项等系统信息；
- watchdog：用于监控文件的读写 / 修改 / 删除等操作。



```
PS C:\snort> pip install psutil watchdog --trusted-host pypi.org --trusted-host files.pythonhosted.org
Python 软件包 psutil-7.2.1-cp37-abi3-win_amd64.whl (136 kB)
136 kB 252 kB/s eta 0:00:01
Downloading watchdog-6.0.0-py3-none-win_amd64.whl (79 kB)
79 kB 1.3 MB/s eta 0:00:01
Installing collected packages: watchdog, psutil
Successfully installed psutil-7.2.1 watchdog-6.0.0
```

图 28 下载实验所需包

20、编写代码，命名为 `show_memory_usage.py`，如下图，导入 `psutil` 包，用于获取当前系统中所有运行的应用程序，显示进程名、PID、占用内存（MB）、内存使用率，按内存占用降序排列



```
1 import psutil
2
3
4 def show_app_memory_usage(): 1 个用法
5     """显示应用程序内存占用情况 (按占用降序)"""
6     # 存储进程内存信息: {进程名: [示例PID, 总占用内存(MB), 最大内存使用率(%)]}
7     process_mem = {}
8
9     # 遍历所有进程, 过滤无权限/无效进程
10    for proc in psutil.process_iter(['pid', 'name', 'memory_info', 'memory_percent']):
11        try:
12            if proc.info['pid'] == 0: # 跳过系统空闲进程
13                continue
14            proc_name = proc.info['name'].lower() # 统一小写避免重复
15            # 物理内存占用 (转换为MB, rss=实际使用的物理内存)
16            mem_mb = round(proc.info['memory_info'].rss / 1024 / 1024, 2)
17            mem_percent = round(proc.info['memory_percent'], 2)
18
19            # 合并同一应用的多进程 (如Chrome多个进程)
20            if proc_name in process_mem:
21                process_mem[proc_name][1] += mem_mb
22                process_mem[proc_name][2] = max(process_mem[proc_name][2], mem_percent)
```

图 29 查看内存的代码

```
C:\Users\30682\AppData\Local\Programs\Python\Python310\python.exe C:\snort\show_memory_usage.py
正在获取应用程序内存占用...
```

应用程序名	PID(示例)	占用内存(MB)	内存使用率(%)
pycharm64.exe	2972	2181.48	26.63
svchost.exe	400	905.0799999999999	0.52
msedgewebview2.exe	5396	694.62	2.64
msedge.exe	3348	407.08000000000004	1.98
mssmpeng.exe	2948	307.33	3.75
searchapp.exe	6968	138.52	1.06
explorer.exe	5028	136.68	1.67
onedrive.exe	8984	130.7	1.6
runtimebroker.exe	2708	121.56	0.4
msspcmanagerservice	3888	96.05	1.17
m365copilot.exe	436	87.0	1.06
powershell.exe	10080	80.4	0.98
registry	108	69.54	0.85
memcompression	1508	67.29	0.82

图 30 应用程序占用内存情况（部分）

21、再编写一个代码文件，命名为 `stat_traffic_10min.py`，统计 10 分钟内各应用的发送流量（上传）和接收流量（下载）（单位：MB），核心是记录初始 / 结束的网络 IO 差值，按总流量降序排列。运行代码后，在 10 分钟内执行以下操作（触发流量）：本地机执行 `telnet 192.168.146.128 23`；实验主机进行浏览网页等让它产生一些流量。

```
1 import psutil
2 import time
3 from datetime import datetime, timedelta
4
5
6 def get_process_net_io(): 2 用法
7     """ 获取当前各进程的网络IO (发送/接收字节) """
8     net_io_dict = {}
9     for proc in psutil.process_iter(['pid', 'name', 'net_io_counters']):
10         try:
11             if proc.info['pid'] == 0 or not proc.info['net_io_counters']:
12                 continue
13             proc_name = proc.info['name'].lower()
14             # 发送字节、接收字节 (原始单位: bytes)
15             send_bytes = proc.info['net_io_counters'].bytes_sent
16             recv_bytes = proc.info['net_io_counters'].bytes_recv
17
18             # 合并同一应用的多进程流量
19             if proc_name in net_io_dict:
20                 net_io_dict[proc_name][0] += send_bytes
21                 net_io_dict[proc_name][1] += recv_bytes
22             else:
23                 net_io_dict[proc_name] = [send_bytes, recv_bytes]
```

图 31 记录流量代码

获取到的进程和流量如图 32、33 所示，系统总发送流量为 5.52MB、接收流量 68.82MB，总流量共 74.34MB；期间有网络活动的应用包含系统服务进程（如 `svchost.exe`）、浏览器进程（如 `msedge.exe`）、

PyCharm 开发工具进程 (pycharm64.exe) 等, 这些应用通过不同的本地 / 远程端口完成了网络连接 (比如 443 端口对应 HTTPS 通信)。

```
PS C:\snort> python stat_traffic_10min.py
===== Windows流量统计 (10分钟) =====
开始时间: 2025-12-29 17:44:28
结束时间: 2025-12-29 17:54:28
初始有网络连接的应用: ['svchost.exe', 'system idle process', 'wininit.exe', 'ctfmon.exe', 'services.exe', 'msedge.exe', 'msedgewebview2.exe', 'system', 'onedrive.exe', 'searchapp.exe', 'lsass.exe', 'spoolsv.exe', 'mpdefendercoreservice.exe']

期间可操作网络 (如telnet、浏览网页、打开QQ) ...

进度: 已统计 1 分钟, 剩余 9 分钟
进度: 已统计 2 分钟, 剩余 8 分钟
进度: 已统计 3 分钟, 剩余 7 分钟
进度: 已统计 4 分钟, 剩余 6 分钟
进度: 已统计 5 分钟, 剩余 5 分钟
进度: 已统计 6 分钟, 剩余 4 分钟
进度: 已统计 7 分钟, 剩余 3 分钟
进度: 已统计 8 分钟, 剩余 2 分钟
进度: 已统计 9 分钟, 剩余 1 分钟
进度: 已统计 10 分钟, 剩余 0 分钟
```

图 32 获取流量

```
=====
【Windows系统10分钟总流量】
发送流量: 5.52 MB | 接收流量: 68.82 MB | 总流量: 74.34 MB
=====
【期间有网络活动的应用】
应用程序名      网络连接 (本地:远程端口)
-----
svchost.exe      62889:0; 5353:0; 500:0
ctfmon.exe       49975:443
system idle proces 49974:443; 49978:443
onedrive.exe     49748:443
lsass.exe        49664:0
wininit.exe      49665:0
mpdefendercoreserv 49977:443
services.exe     49670:0
msedge.exe       50304:443; 50313:443; 50284:443
spoolsv.exe      49668:0
pycharm64.exe    63342:0
searchapp.exe    49681:443; 49685:443; 49679:443
system           137:0; 445:0; 139:0
msedgewebview2.exe 49957:443; 49959:443
```

图 33 获取到的流量

22、再编写一个代码 monitor_file_io.py, 用来监控 C:\snort\rules\new.rules 的读取、修改、删除操作, 实时输出操作类型、时间、操作进程。运行代码后, 手动操作 new.rules

```

1  import time
2  import os
3  from watchdog import observers
4  from watchdog.events import FileSystemEventHandler
5  import psutil
6  from datetime import datetime
7
8
9  class FileMonitor(FileSystemEventHandler): 1 个用法
10     """自定义文件监控处理器"""
11
12     def __init__(self, target_file):
13         self.target_file = target_file # 要监控的文件路径
14
15     def get_op_process(self): 3 用法
16         """获取操作文件的进程"""
17         try:
18             for proc in psutil.process_iter(['pid', 'name', 'open_files']):
19                 if proc.info['open_files']:
20                     for f in proc.info['open_files']:
21                         if self.target_file.lower() in f.path.lower():
22                             return f"PID:{proc.info['pid']} ({proc.info['name']})"
23         except:
24             return "未知进程"

```

图 34 代码编写

23、修改其中内容时，终端会实时显示文件，然后将文件删除或者重新创建，也会显示，如图 35、36。

```

开始监控文件: C:\snort\rules\new.rules | 按Ctrl+C停止

[2025-12-29 18:02:40] 📄 文件修改: C:\snort\rules\new.rules | 操作进程: 未知进程

```



图 35 修改文件

```

开始监控文件: C:\snort\rules\new.rules | 按Ctrl+C停止

[2025-12-29 18:02:40] 📄 文件修改: C:\snort\rules\new.rules | 操作进程: 未知进程
[2025-12-29 18:04:22] 📄 文件修改: C:\snort\rules\new.rules | 操作进程: 未知进程
[2025-12-29 18:04:26] ✖ 文件删除: C:\snort\rules\new.rules | 操作进程: 未知进程
[2025-12-29 18:04:33] ✨ 文件创建: C:\snort\rules\new.rules | 操作进程: 未知进程
[2025-12-29 18:04:38] 📄 文件修改: C:\snort\rules\new.rules | 操作进程: 未知进程

```

图 36 创建和删除文件

24、新建 show_startup_processes.py，用于获取 Windows 开机自启进程 / 服务，显示进程名、自启路径、启用状态，运行代码后，输出系统自启项，如图 38 所示，这个结果展示了实验主机的开机自启进程情况：OneDrive、MicrosoftEdgeAuto、Quark 相关进程属于常用软件的正常注册表自启项，符合日常使用的自启配置；但其中 “akrnozgutxxsav002” 进程的自启路径位于 Downloads 目录，且附带 “密码: hackerhouse” 的描述，属于异常可疑的自启项，是恶意程序的开机启动配置（这是我之前在陈老师的实验课做的病毒实验的残留样本），这也体现了 HIDS 监控系统启动进程的核心价值，及时识别并预警异常自启的风险程序。

```
w_memory_usage.py  stat_traffic_10min.py  monitor_file_io.py  show_startup_processes.py x
1  import psutil
2  import winreg # Windows注册表操作
3
4
5  def get_startup_procs(): 1个用法
6      """获取系统开机自启进程"""
7      startup_list = []
8
9      # 1. 获取自启服务（启动类型为auto）
10     try:
11         for service in psutil.win_service_iter():
12             s = service.as_dict()
13             if s['start_type'] == 'auto' and s['status'] == 'running':
14                 startup_list.append({
15                     "name": s['name'],
16                     "path": s['binary_path'],
17                     "status": "已启用（系统服务）"
18                 })
19     except psutil.AccessDenied:
20         pass
21
22     # 2. 从注册表获取用户自启项
```

图 37 编写代码

```
C:\Users\30682\AppData\Local\Programs\Python\Python310\python.exe C:\snort\show_startup_processes.py
正在获取系统启动进程...
```

进程名	自启路径	状态
OneDrive	"C:\Users\30682\AppData\Local\Microsoft\OneDrive...	已启用（注册表自启）
MicrosoftEdgeAutoL	"C:\Program Files (x86)\Microsoft\Edge\Applicati...	已启用（注册表自启）
QuarkUpdaterTaskUs	"C:\Users\30682\AppData\Local\QuarkUpdater\Quark...	已启用（注册表自启）
quark	"C:\Users\30682\AppData\Local\Programs\Quark\qua...	已启用（注册表自启）
akrnozgutxxsav002	"C:\Users\30682\Downloads\勒索软件样本 密码: hackerhouse\...	已启用（注册表自启）

进程已结束，退出代码为 0

图 38 输出结果

五、实验分析总结及心得

问题 1: 多数 Windows 操作系统防火墙默认会拦截 ICMP 请求, 导致主机在线但 ping 不通, 允许访问的规则默认是被禁用的, 也就是说在实验步骤 5 做包记录实验时, 我尝试用外部主机也就是本机去 ping 虚拟机一开始是 ping 不通的, 如下图所示可以通过修改防火墙策略放行 ICMP 请求。

Win+R 快捷键打开运行, 输入 wf.msc, 点击“入站规则”并修改允许 ICMP 回显。当然, 也可以直接关闭防火墙, 一劳永逸, 但是不安全。

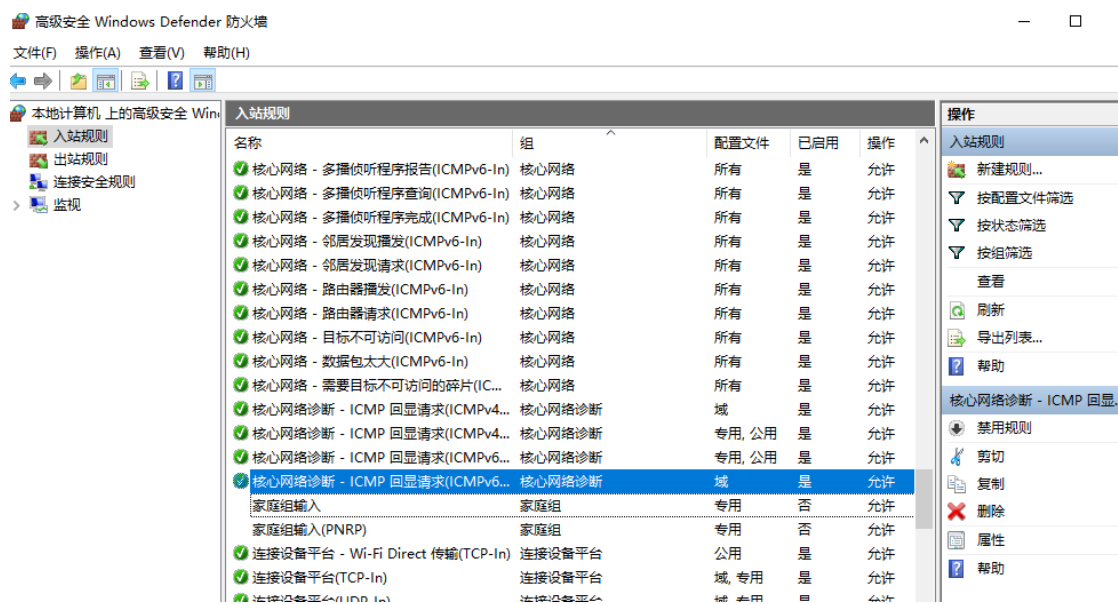


图 39 修改防火墙策略

```
C:\Users\86137>ping 192.168.146.128

正在 Ping 192.168.146.128 具有 32 字节的数据:
请求超时。
请求超时。
请求超时。
请求超时。

192.168.146.128 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 0, 丢失 = 4 (100% 丢失),
```

图 40 ping 不通


```
C:\Users\86137>ping 192.168.146.128

正在 Ping 192.168.146.128 具有 32 字节的数据:
来自 192.168.146.128 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.146.128 的回复: 字节=32 时间=3ms TTL=128
来自 192.168.146.128 的回复: 字节=32 时间<1ms TTL=128
来自 192.168.146.128 的回复: 字节=32 时间<1ms TTL=128

192.168.146.128 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms, 最长 = 3ms, 平均 = 0ms

C:\Users\86137>|
```

图 41 能 ping 通

问题 2: Snort 入侵检测模式需要加载事件消息配置文件（比如 classification.config，用于定义报警分类、优先级等），这些都需要实现配置好，在 snort.conf 中添加引用

```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.19045.6456]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\30682>cd C:\snort\bin

C:\snort\bin>snort -c C:\Snort\etc\snort.conf -i 1 -A console 2>NUL
Could not set the event message file.
C:\snort\bin>snort -c C:\snort\etc\snort.conf -i 1 -A console 2>NUL
Could not set the event message file.
C:\snort\bin>_
```

图 42 找不到文件



图 43 添加事件配置文件

但是解决完这个问题之后，又报了如下所示的错误，看不太懂，询问 AI 后，得知 snort.conf 里包含了 Linux 系统的动态预处理模块路径，而 Windows 没有这个路径，导致 Snort 解析配置失败；同时仍触发 Could not set the event message file，我尝试就是重写一个极简版的 snort.conf，避开 Linux 配置，也不行，又会引发一系列问题，最终解决办法就是直接在命令行运行 snort -i 1 -A console -c C:\Snort\rules\new.rules，就直接用写的规则去检测，不用繁琐的配置了。

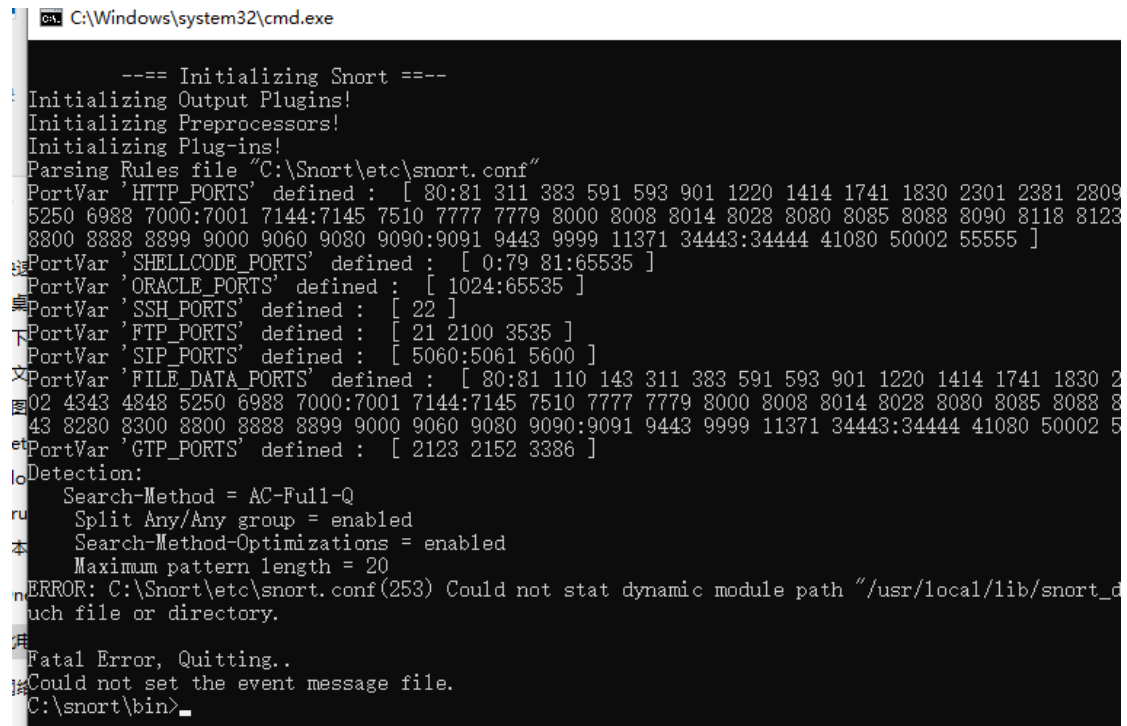


图 44 配置矛盾问题

```

Verifying Preprocessor Configurations:

[ Port Based Pattern Matching Memory ]
pcap DAQ configured to passive.
The DAQ version does not support reload.
Acquiring network traffic from "\\Device\\NPF_{C793FC0A-DA56-4F84-BC9D-C3C50C2989E3}"
Decoding Ethernet

    === Initialization Complete ===

    o''')~  -> Snort! <*-
    ,,,~   Version 2.9.16-WIN32 GRE (Build 118)
           By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2020 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using PCRE version: 8.10 2010-06-25
           Using ZLIB version: 1.2.3

Commencing packet processing (pid=9000)
WARNING: No preprocessors configured for policy 0.

```

图 45 成功运行

总结与心得：总的来说，学会了 snort 这个工具的大致使用方法，主要就是抓包，跟我在陈老师实验课中使用的一个软件 wireshark 差不多，也是抓包工具，但是它更方便的一点就是能设定规则去触发告警信息，尤其是能自己设定规则，大大扩展了它的使用空间和便利性，在实验中，我也尝试自己编写规则然后在本机上用 telnet 尝试连接时，真的触发 alert 信息了，后期还能通过编写代码将告警信息发送到自己的 qq 邮箱，对防御网络攻击有了新的认识吧。然后在实验中也遇到了许多问题，感觉 snort 这个软件的博客比较少，在遇到版本不匹配尤其是那个 rules 文件使用时报了很多很多的问题，查不到准确的解决方法博客，最终在 AI 的建议下使用简化的方法，在命令行直接加载规则文件而不使用它那个 conf 配置文件，最终成功做出来了，也是很有成就感的。